

High Performance Computing

OpenMP Lab Assignment

Installing and Using OpenMP

Before starting the exercises, make sure your compiler supports OpenMP.

Linux

Install the GNU C++ compiler if it is not already installed:

```
sudo apt update
sudo apt install g++
```

OpenMP support is included with recent versions of GCC. Compile OpenMP programs using the `-fopenmp` flag:

```
g++ -O2 -fopenmp program.cpp -o program
```

Windows

Install MSYS2 with the MinGW-w64 GCC toolchain or use Microsoft Visual Studio with C++ development tools. When using GCC, compile with:

```
g++ -O2 -fopenmp program.cpp -o program.exe
```

For Visual Studio, enable OpenMP support in the project settings.

macOS

Install LLVM using Homebrew:

```
brew install llvm libomp
```

Compile using the LLVM compiler with OpenMP enabled:

```
clang++ -Xpreprocessor -fopenmp program.cpp \
-lomp -o program
```

Running OpenMP Programs

You can control the number of threads by setting the environment variable:

```
export OMP_NUM_THREADS=4
```

or directly in your program:

```
1 #include <omp.h>
2
3 omp_set_num_threads(4);
```

Objectives

The purpose of this exercise is to become familiar with basic shared-memory parallel programming using OpenMP. You will create OpenMP threads, parallelize loops, examine how work is divided among threads, use reduction to combine partial results, experiment with different scheduling strategies, and observe a simple race condition.

Part 1: Your First OpenMP Program

Consider the following program:

```
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5
6     #pragma omp parallel
7     {
8         int id = omp_get_thread_num();
9         int nthreads = omp_get_num_threads();
10
11         std::cout << "Hello from thread "
12                 << id
13                 << " of "
14                 << nthreads
15                 << std::endl;
16     }
17
18     return 0;
19 }
```

Compile the program using:

```
g++ -O2 -fopenmp openmp_test.cpp -o openmp_test
```

Run the program several times.

You may control the number of threads using:

```
export OMP_NUM_THREADS=4
```

or by using:

```
1 omp_set_num_threads(4);
```

Try the program with:

1, 2, 4

threads.

```

part1.cpp X part2.cpp part3.cpp part4.cpp part5.cpp part6.cpp part7.cpp
part1.cpp > main()
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5     omp_set_num_threads(4);
6
7     #pragma omp parallel
8     {
9         int id = omp_get_thread_num();
10        int nthreads = omp_get_num_threads();
11
12        std::cout << "Hello from thread "
13                << id
14                << " of "
15                << nthreads
16                << std::endl;
17    }
18
19    return 0;
20 }

```

Problems Output Debug Console Terminal Ports

inal will be reused by tasks, press any key to close it.

- PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
Hello from thread Hello from thread 3 of 4Hello from thread 1 of 4
0 of 4
Hello from thread 2 of 4
- PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
Hello from thread 0 of 1
- PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
Hello from thread Hello from thread 01 of 2 of 2

Ln 6, Col 1 Spaces: 4 UTF-8

```

part1.cpp X
part1.cpp > main()
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5     omp_set_num_threads(4);
6
7     #pragma omp parallel
8     {
9         int id = omp_get_thread_num();
10        int nthreads = omp_get_num_threads();
11
12        std::cout << "Hello from thread "
13                << id
14                << " of "
15                << nthreads
16                << std::endl;
17    }
18
19    return 0;
20 }

```

Problems Output Debug Console Terminal Ports

ng\Lab4_AbdanurAyazbek_BDA2406\part1.exe

- PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
Hello from thread 1 of 4Hello from thread 2 of 4
Hello from thread 3 of 4
Hello from thread 0 of 4
- PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
Hello from thread 0 of 1
- PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
Hello from thread Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4
Hello from thread 0 of 4
- PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
Hello from thread Hello from thread 01 of 2 of 2

1 of 4

Ln 15, Col 30 Spaces: 4 UTF-8 CRLF C++ 8-9 Go Live Win32

Answer the following questions.

1. How many times is the code inside the parallel region executed when four threads are used?

4 times

2. What values are returned by `omp_get_thread_num()`?

0, 1, 2, 3

3. Does the order in which the threads print their messages remain the same between different executions?

No, the order is different every time

4. Is the code before and after the parallel region executed by every thread or only by the original thread?

Only by the original thread

Part 2: Parallel Vector Computation

Consider two vectors A and B containing N elements.
For every element, calculate

$$C_i = 2A_i + B_i.$$

Use

$$N = 1,000,000.$$

Start with the following program:

```
1 #include <iostream>
2 #include <vector>
3 #include <omp.h>
4
5 int main() {
6
7     const int N = 1000000;
8
9     std::vector<double> A( N);
10    std::vector<double> B( N);
11    std::vector<double> C(N);
12
13    for (int i = 0; i < N; ++i) {
14        A[i] = i * 0.5;
15        B[i] = i * 0.25;
16    }
17
18    // Parallelize this loop
19    for (int i = 0; i < N; ++i) {
20        C[i] = 2.0 * A[i] + B[i];
21    }
22
23    std::cout << "C[0] = " << C[0] << std::endl;
24    std::cout << "C[N-1] = " << C[N-1] << std::endl;
25
26    return 0;
27 }
```

Modify the program so that the computation of vector C is performed using an OpenMP parallel loop.

```
part2.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <omp.h>
4
5  int main() {
6      const int N = 1000000;
7
8      std::vector<double> A(N);
9      std::vector<double> B(N);
10     std::vector<double> C(N);
11
12     for (int i = 0; i < N; ++i) {
13         A[i] = i * 0.5;
14         B[i] = i * 0.25;
15     }
16
17     #pragma omp parallel for
18     for (int i = 0; i < N; ++i) {
19         C[i] = 2.0 * A[i] + B[i];
20     }
21
22     std::cout << "C[0] = " << C[0] << std::endl;
23     std::cout << "C[N-1] = " << C[N-1] << std::endl;
24
25     return 0;
26 }
```

Problems Output Debug Console Terminal Ports

ess any key to c lose it.

PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part1.exe
PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part2.exe
C[0] = 0
C[N-1] = 1.25e+06
PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406>

Ln 26, Col 2 Spaces: 4 UTF-8

Questions

1. Can different iterations of this loop be executed independently?

Yes, each iteration is independent

2. Which OpenMP directive did you use?

#pragma omp parallel for

3. Is the loop variable *i* shared between threads or private to each thread?

Private, each thread has its own copy of **i**

4. Are arrays *A*, *B*, and *C* shared by the threads?

Yes, A,B and C are shared between threads

5. Why is this loop suitable for parallel execution?

Because each element *C[i]* depends only on *A[i]* and *B[i]*, not on any other iteration

Part 3: Observing Work Distribution

Modify the parallel loop so that each iteration also records which thread processed it.

For this experiment, use a smaller value:

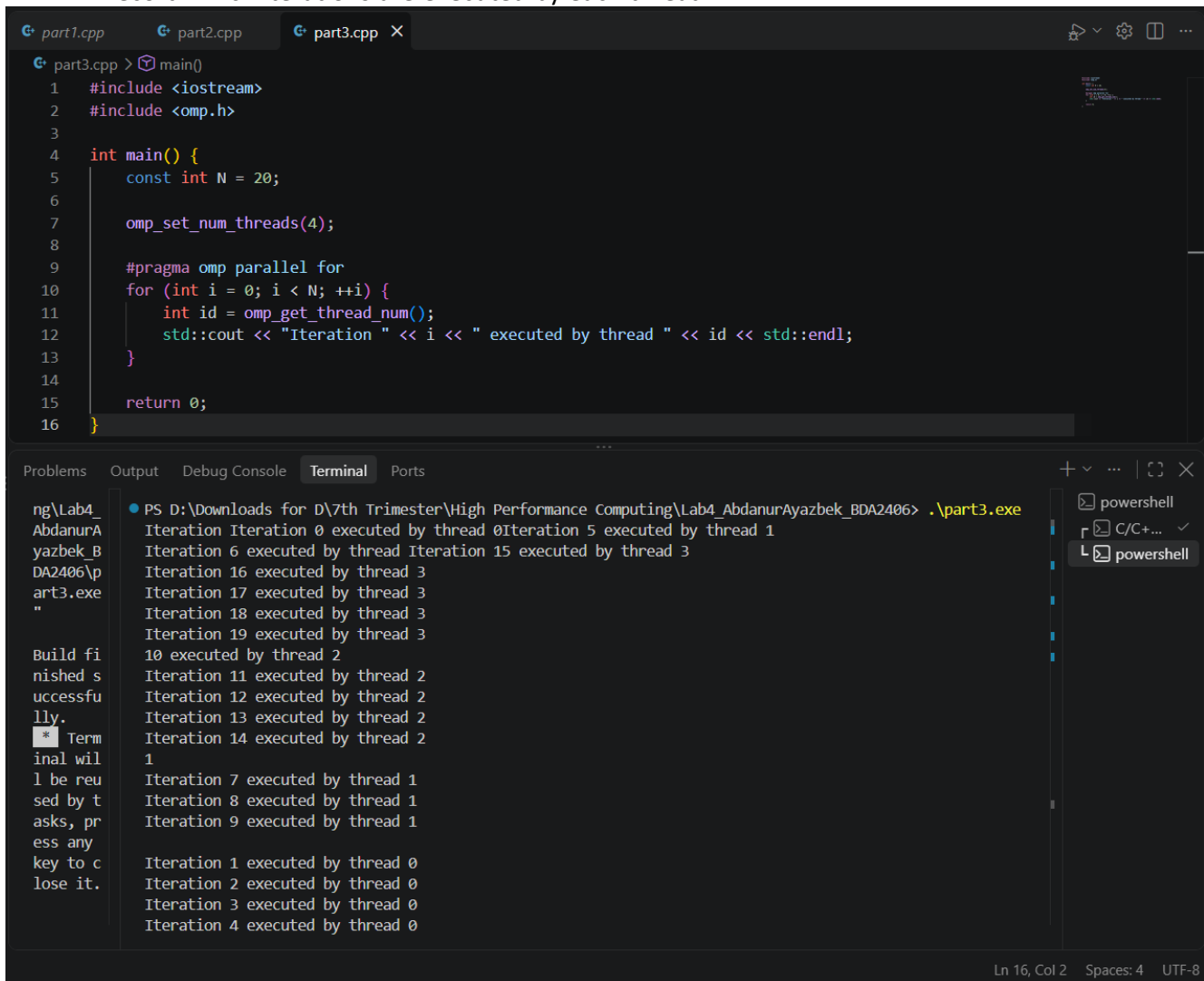
$$N = 20.$$

For example:

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; ++i) {
3
4     int id = omp_get_thread_num();
5
6     std::cout << "Iteration "
7               << i
8               << " executed by thread "
9               << id
10              << std::endl;
11 }
```

Run the program with four threads.

Record which iterations are executed by each thread.



```
part3.cpp > main()
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5     const int N = 20;
6
7     omp_set_num_threads(4);
8
9     #pragma omp parallel for
10    for (int i = 0; i < N; ++i) {
11        int id = omp_get_thread_num();
12        std::cout << "Iteration " << i << " executed by thread " << id << std::endl;
13    }
14
15    return 0;
16 }
```

Terminal Output:

```
PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part3.exe
Iteration 0 executed by thread 0
Iteration 1 executed by thread 0
Iteration 2 executed by thread 0
Iteration 3 executed by thread 0
Iteration 4 executed by thread 0
Iteration 5 executed by thread 1
Iteration 6 executed by thread 1
Iteration 7 executed by thread 1
Iteration 8 executed by thread 1
Iteration 9 executed by thread 1
Iteration 10 executed by thread 2
Iteration 11 executed by thread 2
Iteration 12 executed by thread 2
Iteration 13 executed by thread 2
Iteration 14 executed by thread 2
Iteration 15 executed by thread 3
Iteration 16 executed by thread 3
Iteration 17 executed by thread 3
Iteration 18 executed by thread 3
Iteration 19 executed by thread 3
```

Thread	Iterations
0	0, 1, 2, 3, 4
1	5, 6, 7, 8, 9
2	10, 11, 12, 13, 14
3	15, 16, 17, 18, 19

Answer:

1. Is every iteration executed exactly once?

Yes, every iteration is executed exactly once

2. Does every thread receive approximately the same amount of work?

Yes, each thread gets 5 iterations - equal amount

3. Are consecutive iterations generally assigned to the same thread when the default scheduling strategy is used?

Yes, consecutive iterations are assigned to the same thread with default scheduling

Part 4: Parallel Reduction

Consider the following serial calculation:

$$S = \sum_{i=0}^{N-1} A_i$$

The serial implementation is:

```

1 double sum = 0.0;
2
3 for (int i = 0; i < N; ++i) {
4     sum += A[i];
5 }

```

A direct parallelization such as

```
1 double sum = 0.0;
2
3 #pragma omp parallel for
4 for (int i = 0; i < N; ++i) {
5     sum += A[i];
6 }
```

is not safe because several threads may update sum simultaneously.

Use the OpenMP reduction clause to calculate the sum correctly in parallel.

Complete the following code:

```
1 double sum = 0.0;
2
3 #pragma omp parallel for _____
4 for (int i = 0; i < N; ++i) {
5     sum += A[i];
6 }
```

Test your implementation using

$$A_i = 1$$

for every element and

$$N = 1,000,000.$$

The expected result is therefore

$$S = 1,000,000.$$

```
part4.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <omp.h>
4
5  int main() {
6      const int N = 10000000;
7
8      std::vector<double> A(N, 1.0);
9
10     double sum = 0.0;
11
12     #pragma omp parallel for reduction(+:sum)
13     for (int i = 0; i < N; ++i) {
14         sum += A[i];
15     }
16
17     std::cout << "Sum = " << sum << std::endl;
18
19     return 0;
20 }
```

Terminal: PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part4.exe
Sum = 1e+06

Questions

1. Why is updating one shared sum variable directly unsafe?

Because multiple threads read and write the same variable at the same time. One thread can overwrite another's result before it is saved

2. What does the OpenMP reduction operation do conceptually?

Each thread computes its own local sum, then all local sums are added together at the end

3. Does every thread initially work with the same shared accumulator or with its own partial result?

Each thread works with its own partial result

4. When are the partial results combined?

At the end, after all threads finish their local computation

Part 5: Race Condition

Consider the following program:

```
1  #include <iostream>
2  #include <omp.h>
3
4  int main() {
5
6      const int N = 1000000;
7      int counter = 0;
8
9      #pragma omp parallel for
10     for (int i = 0; i < N; ++i) {
11         counter++;
12     }
```

4
5
6
7
8
9
10
11
12
13

```

14     std::cout << "Counter = "
15             << counter
16             << std::endl;
17
18     return 0;
19 }

```

Run the program several times using four threads.
The expected value is

1,000,000.

Record several results.

Run	Without atomic	With atomic
1	1000000	1000000
2	1000000	1000000
3	1000000	1000000

Now protect the increment operation using:

```

1 #pragma omp atomic
2 counter++;

```

Run the program again.

The screenshot shows the Visual Studio Code editor with a C++ file named `part5.cpp`. The code is as follows:

```

1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5     const int N = 1000000;
6     int counter = 0;
7
8     #pragma omp parallel for
9     for (int i = 0; i < N; ++i) {
10         counter++;
11     }
12
13     std::cout << "Without atomic: " << counter << std::endl;
14
15     counter = 0;
16
17     #pragma omp parallel for
18     for (int i = 0; i < N; ++i) {
19         #pragma omp atomic
20         counter++;
21     }
22
23     std::cout << "With atomic: " << counter << std::endl;
24
25     return 0;
26 }

```

The terminal output shows the results of running the program three times:

```

PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part5.exe
Without atomic: 1000000
With atomic: 1000000
PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part5.exe
Without atomic: 1000000
With atomic: 1000000
PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part5.exe
Without atomic: 1000000
With atomic: 1000000
PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406>

```

Questions

1. Why can the original program produce an incorrect value?

Multiple threads increment counter at the same time. One thread can read the value, another thread changes it, and the first thread overwrites with an outdated value

2. Why is the statement

```
1 counter ++;
```

not necessarily one indivisible operation?

It is three operations: read, add 1, write back. Another thread can interrupt between these steps.

3. What changes after adding `#pragma omp atomic`?

Each increment becomes one indivisible operation. No thread can interrupt it

4. Does synchronization have a performance cost?

Yes, atomic makes every increment wait its turn, which slows down execution

Part 6: Static and Dynamic Scheduling

OpenMP allows the programmer to control how loop iterations are assigned to threads.

Consider the following computation:

```
1 #include <cmath>
2
3 for (int i = 0; i < N; ++i) {
4
5     int work = 100 + (i % 1000);
6
7     double value = 0.0;
8
9     for (int k = 0; k < work; ++k) {
10         value += std::sqrt(k + 1.0);
11     }
```

```

12     result[i] = value;
13 }
14

```

Parallelize this loop first using

```

1 #pragma omp parallel for schedule(static)

```

and then using

```

1 #pragma omp parallel for schedule(dynamic, 10)

```

Use

$N = 100000$.

Measure the execution time of both versions using `omp_get_wtime()`:

```

1 double start = omp_get_wtime();
2
3 // computation
4
5 double finish = omp_get_wtime();
6
7 std::cout << "Time = "
8           << finish - start
9           << " seconds"
10          << std::endl;

```

Complete the table.

Scheduling Method	Execution Time (seconds)
Static	0.016
Dynamic	0.039

Scheduling Method	Execution Time (seconds)
Static	
Dynamic	

Answer:

1. What does `schedule(static)` do?

Divides iterations into equal blocks and assigns them to threads upfront before execution starts

2. What does `schedule(dynamic,10)` do?

Assigns chunks of 10 iterations to threads one at a time as they become free

3. Why might dynamic scheduling be useful when different iterations require different amounts of work?

Because a free thread immediately gets new work instead of waiting for others to finish their large block

4. Does dynamic scheduling introduce additional scheduling overhead?

Yes, because threads constantly ask for new chunks instead of knowing their work upfront

Part 7: Performance Experiment

Return to the vector computation from Part 2.

Increase the problem size if necessary so that the execution time can be measured reliably.

Measure the parallel computation using:

1, 2, 4

threads.

Record your results.

```
part7.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <omp.h>
4
5  int main() {
6      const int N = 10000000;
7
8      std::vector<double> A(N);
9      std::vector<double> B(N);
10     std::vector<double> C(N);
11
12     for (int i = 0; i < N; ++i) {
13         A[i] = i * 0.5;
14         B[i] = i * 0.25;
15     }
16
17     for (int threads : {1, 2, 4}) {
18         omp_set_num_threads(threads);
19
20         double start = omp_get_wtime();
21
22         #pragma omp parallel for
23         for (int i = 0; i < N; ++i) {
24             C[i] = 2.0 * A[i] + B[i];
25         }
26
27         double finish = omp_get_wtime();
28         std::cout << "Threads: " << threads << " Time: " << finish - start << " seconds" << std::endl;
29     }
30
31     return 0;
32 }
```

key to c
lose it.

PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab4_AbdanurAyazbek_BDA2406> .\part7.exe

Threads: 1 Time: 0.0149999 seconds
Threads: 2 Time: 0.0150001 seconds
Threads: 4 Time: 0.0179999 seconds

Ln 16, Col 1 Spaces: 4 UTF-8

Threads	Execution Time	Speedup
1	0.015	1.00
2	0.015	1.00
4	0.018	0.83

Threads	Execution Time	Speedup
1		1.00
2		
4		

Calculate the speedup using

$$S_p = \frac{T_1}{T_p}$$

where T_1 is the execution time using one thread and T_p is the execution time using p threads.

Answer:

1. Does execution time decrease when more threads are used?

No, execution time did not decrease

2. Is the speedup exactly equal to the number of threads?

No, speedup is not equal to the number of threads

3. Give one reason why a program may not obtain ideal speedup.

The task is too small and fast. The overhead of creating and managing threads takes more time than the actual computation

4. Would you expect increasing the number of threads indefinitely to continue improving performance? Explain briefly.

No. At some point adding more threads increases overhead more than it reduces computation time, and performance starts to drop

Part 8: Short Discussion

Answer each question briefly.

1. What is the difference between a serial region and a parallel region in OpenMP?

serial region is executed by one thread. A parallel region is executed by multiple threads simultaneously

2. What is the purpose of `#pragma omp parallel for`?

It splits a loop into parts and runs each part on a different thread at the same time

3. What is the difference between a shared variable and a private variable?

A shared variable is one copy accessed by all threads. A private variable is a separate copy for each thread

4. Why is a reduction useful for operations such as summation?

Because without reduction each thread would overwrite the same variable. Reduction gives each thread its own partial result and combines them safely at the end

5. What is a race condition?

When two threads read and write the same variable at the same time and the result depends on which thread goes first

6. Why can synchronization reduce parallel performance?

Because threads have to wait their turn instead of running freely, which removes the benefit of parallelism

7. What is the difference between static and dynamic scheduling?

Static assigns equal blocks to threads upfront. Dynamic assigns small chunks one at a time as threads become free

Conclusion

In this lab I learned the basics of OpenMP parallel programming. I created parallel regions, parallelized loops, used reduction to safely sum results, observed a race condition, and compared static vs dynamic scheduling. The performance experiment showed that parallelism does not always improve speed since thread overhead can outweigh the benefit for small tasks.

Submission

Submit:

1. all C++ source files used in the exercise;
2. output from the first OpenMP program;
3. the completed work-distribution table;
4. the reduction result;
5. the race-condition experiment results before and after synchronization;
6. execution times for static and dynamic scheduling;
7. the performance and speedup table;
8. answers to all questions.